

Ceiling Function

Day 2 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“store them in a binary search tree”*
- ▶ *“trees of the same shape”*

Programming interpretation:

- ▶ Each prototype is a sequence of insertions into a BST.
- ▶ The actual numeric labels do not matter after insertion; only left/right shape matters.
- ▶ Serialize each shape into a tuple and count unique tuple shapes.

Textbook connection: CP4 2.3.3 Balanced BST / 3.2 Search Modeling

Core technique: BST construction and structural hashing

- ▶ Each sequence is inserted into a binary search tree in input order.
- ▶ Only the shape of the resulting tree matters.
- ▶ The answer is the number of distinct tree shapes.

- ▶ Different values can create the same comparison structure.
- ▶ If two trees have the same left/right recursive shape, they are equivalent.
- ▶ A set can store canonical shape descriptions and remove duplicates.

- ▶ For each sequence, create a root from the first value.
- ▶ Insert remaining values using ordinary BST rules.
- ▶ Convert the finished tree to a recursive tuple shape.
- ▶ Use None for empty children and ignore stored values.
- ▶ Insert the tuple into a set and print the set size.

- ▶ `get_shape(root)` returns `(left_shape, right_shape)`.
- ▶ Leaf nodes become `(None, None)`.
- ▶ Two equal tuples mean the same exact branching structure.

- ▶ BST insertion is simulated exactly for every sequence.
- ▶ The tuple representation records every left and right child position.
- ▶ Ignoring values is valid because the problem compares only tree shape.

Complexity: $O(N * K^2)$ worst-case time for N sequences of length K ; $O(N * K)$ memory for stored shapes.

- ▶ The tree may be unbalanced, but the input bounds make this acceptable.
- ▶ Tuples are hashable, so they can be stored directly in a Python set.

Source lines: ceiling.py:32-40

```
import sys

class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

What this chunk does

- ▶ Each node stores the inserted value and pointers to left and right children.
- ▶ None children are important because they distinguish tree shapes.

Source lines: ceiling.py:42-55

```
def insert(root, value):  
    if value < root.value:  
        if root.left is None:  
            root.left = Node(value)  
        else:  
            insert(root.left, value)  
    else:  
        if root.right is None:  
            root.right = Node(value)  
        else:  
            insert(root.right, value)
```

What this chunk does

- ▶ Smaller values go left; all other values go right.
- ▶ Insertion order is what creates the final shape.
- ▶ The code recurses until it finds the empty child position.

Source lines: ceiling.py:57-64

```
def get_shape(root):  
    if root is None:  
        return None  
  
    return (get_shape(root.left), get_shape(root.right))
```

What this chunk does

- ▶ Empty children become None.
- ▶ A real node becomes a pair containing the shapes of its left and right subtrees.
- ▶ Values are intentionally ignored, so equal structures produce equal tuples.

Source lines: ceiling.py:66-89

```
def main():
    input = sys.stdin.buffer.readline

    n, k = map(int, input().split())

    shapes = set()

    for _ in range(n):
        values = list(map(int, input().split()))

        root = Node(values[0])

        for value in values[1:]:
            insert(root, value)

        shapes.add(get_shape(root))

    print(len(shapes))

main()
```

What this chunk does

- ▶ Each prototype builds its own BST from the given sequence.
- ▶ The serialized shape is inserted into a set, which removes duplicates.
- ▶ The number of set entries is the answer.

Solution file: `practice_contests/day_2/ceiling.py`

Problem: <https://open.kattis.com/problems/ceiling>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.